Trajectory-Informed Metacognitive Scaffolding for Programming: Choosing the Moment, and Staying Inside the Evidence

Minahil Kashif
Project G3-11, Roll No. 23i-0554
National University of Computer and Emerging Sciences
Islamabad, Pakistan
i230554@isb.nu.edu.pk

Areeba Javed
Project G3-11, Roll No. 23i-0570
National University of Computer and Emerging Sciences
Islamabad, Pakistan
i230570@isb.nu.edu.pk

***Abstract*—In the age of generative AI, students may complete programming tasks with no planning, supervision or reflection. Code assistants generate valid syntax in response to a single prompt. Novice programmers avoid challenging problem-solving steps such as decomposition, hypothesis generation and verification. Reflective questioning conversational tutoring agents attempt to recover this by asking reflective questions. However, the current solutions have three major issues. First, the systems interrupt while the student is typing. Second they make unsupported claims about the code. Finally they measure success on speed of task completion rather than learning. Here we present TrajectoryCoach, a novel agentic scaffolding tool that uses metacognitive reasoning based on the student's trajectory. The system uses three mechanisms to prevent coding. A temporal inactivity gate (G1) delays prompting until the student has paused naturally for 20 seconds. An evidence minimum (G2) ensures that no prompting can happen unless there are five events observed, enforcing strict JSON-schema validation. Finally, a delayed unassisted transfer (G3). The system is evaluated by replaying 726,897 IDE events from eight held-out novice learners. There are 10,687 observable decision points. The temporal inactivity gate reduces interruptions during typing bursts to 0.021 compared to 0.127 for on-demand, pause-locked prompting and 0.657 for the non-gated Phase 1 policy. The disadvantage of this approach is that it reduces the coverage of decisions made to 0.137 compared to 0.793 for the non-gated rule baseline. When looking at 100 identical trajectory prefixes using the same model, only 0.14 of prompts were flagged as making unsupported claims with prompt v1 (0.53 non-gated), whilst with prompt v2 (written after seeing what was flagged with v1) none were flagged (0.55 non-gated). Delayed unassisted transfer shows slight improvements in correlation with latent understanding ($\rho = 0.793 \pm 0.036$) compared to immediate assisted completion ($\rho = 0.724$) when simulating cohorts of learners. As we are simply replaying logs, these figures represent how selective the policy is, rather than showing improved learning.**

***Index Terms*—metacognition, programming education, agentic systems, intervention timing, evidence grounding, learning analytics**

I. Introduction

Nowadays students are abusing generative AI tools. How are we preparing students for the future of cognitive and problem-solving? A real problem is that students are using generative technologies to complete almost all assignments/tasks within their academic work, without any analysis, planning, monitoring and evaluation. This has led to cognitive offloading, whereby external tools are used to reduce the amount of mental effort needed for a task. Especially in programming students can use lots of tools to make programming extremely easy. For example a first year student can easily complete an introductory assignment if their commercial coding copilot can produce a clean python function from a single line of text. But how fast is this? Without breaking the problem down, testing and self diagnosing errors students will have weak mental models [1, 2] and will fail exams [3] with no help.

Attempts have been made to use automated tutoring agents to make students reflect. However, integrating an assistive tool into a real code editor can lead to:

1) Proactive tutors are more likely to appear mid-thought (Gap 1). This causes the student to be interrupted and are not happy with the prompt [?] [4].
2) Ungrounded Hallucinations (Gap 2). Language models often suggest syntax/logic suggestions that are not grounded in the telemetry they are given. Students cannot see this and are therefore tricked.
3) Shallow Evaluation (Gap 3). Most studies evaluate the prompt based on how quickly the student can complete an exercise with the help of AI. Therefore they associate quick results [5].

To overcome these issues we have developed TrajectoryCoach, an agentic intervention system. It will provide metacognitive prompts based on evidence during the 3 stages of learning (planning, monitoring and evaluating) (Fig. 1). It will not generate code, or read the students source code. It will just use coarse edit and run telemetry. The system uses behavioural rhythm and will never analyse any code, therefore we consider the students source code as private.

Our design includes:

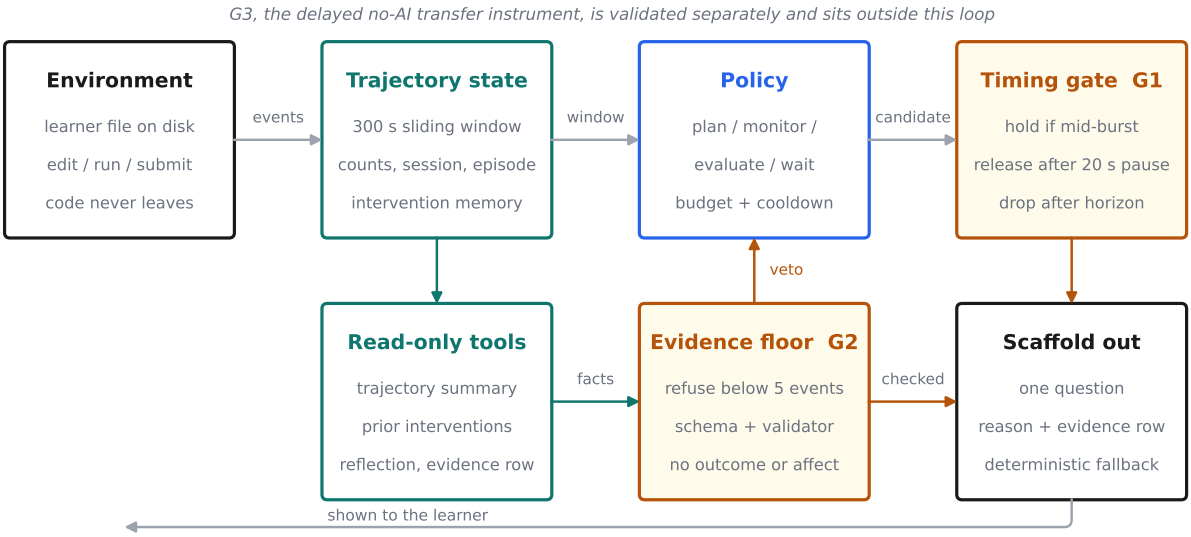


Fig. 1. TrajectoryCoach. Events from the learner's file become a bounded trajectory state; the policy proposes a scaffold; the timing gate (G1) decides *when* it is delivered and the evidence floor and validator (G2) decide *whether* and *in what terms*. The agent reaches the state only through read-only tools. G3, the delayed transfer instrument, sits outside this loop.

An Inactivity Timing Gate (G1). This will store the candidate prompt whilst a decision junction is triggered. The prompt will only be released if the learner has been inactive for 20s+. Whilst they are typing the prompt will still be there. If they do not have a pause within 600s the prompt will be dropped.

Evidence Floor and Validation Contract (G2). The policy will not speak unless there are at least 5 events. The prompts must be created using a language model with a very strict JSON schema. They are then checked by an external validator. The model is told not to mention any outcomes, feelings or code. Then the prompts are scored by a claim checker.

To measure how effective the scaffolding is we have developed a delayed transfer task (unassisted) using a 4 point rubric (G3). We have validated this using simulated learners.

We tested TrajectoryCoach by replaying 726,897 events from the IDE of 8 held out learners in an introductory programming MOOC (CS1). The timing gate reduced the interruption rate to 0.021. This is significantly lower than on demand assistance (0.127 - simulated requests at natural pauses) and an ungated Phase 1 policy (0.657). During live generation 14/100 prompts were flagged for ungrounded claims using a prompt v1. Whereas 0/100 prompts using a prompt v2. Compared to 53/100 and 55/100 prompts using the same model with no constraints. It can handle 29 366 events per second with only 1.5MB of RAM needed (offline replay mode) with only one CPU core.

II. Related Work

A. Problem Solving in Programming

Students are often taught low level syntax. However, planning, monitoring and evaluating are metacognitive skills that are taught to novices [6]. Using tutoring systems has been shown to be an effective way to help learners learn by providing attention [Wood et al.?]. Initial attempts at providing early computer work were to develop worksheets and static prompts to encourage reflection [7]. More recent systems generate reflective questions based on the context [1, 8]. The issue with these models is that they become increasingly specific if they are not constrained. This leads to actual code being written. However this is not the intention with self directed learning.

B. Proactive vs Requested Help

One of the main issues discussed in intelligent tutoring systems is who should be in control of the interventions. Requested help (on demand) gives the learner control however comes with some issues. Students with the most need do not ask for help most. Students with the highest academic levels do ask for help but only to get the answer quickly [5]. Uncoordinated interruptions cause working memory loss and proactive systems take over. Sudden interruptions cause several minutes of reflection for the programmer. The interruption effect is much less if it is delayed until a task boundary or natural pause [4, ?]. We have used this principle during programming education. It provides a good time to start a conversation whilst the student is inactive.

C. Performance vs Learning

If AI tutors are judged on how many tasks can be helped with then a false perception of how useful they are will be created. This has been found in automated feedback systems [9, 10]. Students who complete an AI co-pilot exam quickly will perform badly on a non-AI exam [3]. Real learning requires independent knowledge retrieval and application. Our

assessment tasks have been designed to be unassisted transfer tasks.

D. Grounded Language Models in Education

LLM's hallucinate meaning they say things that sound correct but are incorrect [?]. This has been a bigger issue in education as beginners will not understand the difference. The advice will not be grounded and will only confuse them further [11] [12]. Common interventions used are prompt steering, retrieval augmented generation (RAG) and agentic tool use [?][13]. However we have explicitly checked each candidate prompt against an evidence contract. This is done automatically using a JSON schema, linked to a row in the telemetry log.

E. Ethical Issues with Telemetry

Monitoring students is a real issue regarding privacy [?]. Some research tutors will log all of the audio, web history and even webcam. TrajectoryCoach takes the opposite approach and instead tries to limit observation. We do not use any source code, variable names or student names. Only anonymised timestamps for edit events, focus changes and run/submit events [14].

III. Problem Formulation

A. Telemetry Data

We can represent the actions of students within the IDE as a list of events, which can only be appended to:

$$S = \langle e_1, e_2, \ldots, e_T \rangle \quad \text{where} \quad e_k = (t_k, a_k, f_k) \tag{1}$$

Where the inputs are the time of the event (t_k), the action type (text_insert, text_remove, text_paste, focus_gained, focus_lost, run, submit) (a_k) and the task (as identified by part and exercise) (f_k). An anonymised identifier and source row is also provided.

B. Maximum Exposures

To safeguard student data and reduce overhead we remove some commonly used data channels in tutoring systems:

No source code examination: Do not examine student's text, abstract syntax tree or variable names.

No execution examination: Cannot examine normal output, error tracebacks or terminal logs.

No ground Truth Correctness: There is no automatic marking or unit test outcomes associated with the policy.

C. Metacognition Phases

The scaffolding is split into three phases of problem solving:

1) *Plan:* At the start of the task, or the first time the student begins to code. Ask the student to explain what their goal is and provide an example input/output pair.
2) *Monitor:* When the student has executed code 3+ times in the last 300 seconds. Edits made at this phase. Ask the student to explain what they're testing and what evidence would convince them.
3) *Evaluate:* When a formal submission is made. Ask the student to compare their code with edge cases.

D. Research Questions

We will investigate the following research questions:

RQ1: Is it possible to reduce the interruption frequency during typing bursts by using a timeout gate and have more decision points than with on-demand?

RQ2: Does having a minimum evidence requirement and validation contract for a structured document reduce unfounded claims (outcome, feelings, code and unseen numbers) made by the LM?

RQ3: Does providing a no-assistance delayed transfer task provide a better distinction between real and non-real problem solving skills than an instantaneous assisted completion with simulated learners?

IV. System Design

A. Trajectory State

TrajectoryCoach maintains a sliding window state of the telemetry stream S (last 300 seconds). It calculates some summary statistics including Number of times code has been run (N_{run}), Number of edits made (N_{edit}), Number of times submitted (N_{sub}) and Time since last event (Δt_{idle}). It will also have information about the current task, current session (new session if no activity for $>$30 mins) and any prompts that have been given.

B. Deterministic Rules

There are three deterministic rules within the policy to avoid students becoming "prompt fatigued" whilst still allowing autonomy:

Timeout (180s) - Once a scaffold is given there will be a 180s break.

Episode Limit (3) - Up to 3 prompts can be given per task (resets on new session).

Wait (120s) - Once an intervention has been given the system will wait up to 120s for a written reflection.

C. Timeout Gate (G1)

The interruption is controlled using a timeout gate. TrajectoryCoach will not show the dialog box when the state machine identifies a candidate decision point j. Instead, it will add the Candidate prompt to a waiting queue:

$$\text{Gate}(j,t) = \begin{cases} \text{Deliver}, & \text{if } t - t_{\text{last_event}} \geq \tau_{\text{pause}} \\ \text{Hold}, & \text{if } t - t_{\text{last_event}} < \tau_{\text{pause}} \\ & \text{and } t - t_{\text{juncture}} \leq H_{\max} \\ \text{Drop}, & \text{if } t - t_{\text{juncture}} > H_{\max} \end{cases} \tag{2}$$

Where $\tau_{\text{pause}} = 20$s is the timeout threshold and $H_{\max} = 600$s is the maximum amount of time the prompt can be stored. If the student types in under 20s then the prompt will remain in the buffer. If a new candidate occurs then the previous candidate will be replaced. If the student doesn't pause at all and types for the full 600s then the scaffold will be silently removed.

D. Minimum Evidence Requirement and Validation Contract (G2)

We assume the LM is not reliable when generating prompts. As such, every candidate prompt must satisfy a clear evidence contract:

$$C = \{\text{phase} \in \{\text{plan}, \text{monitor}, \text{evaluate}\}, \text{reason} \in E, \text{text} \in T\} \tag{3}$$

Where E is an evidence object from the recent telemetry. A JSON schema is used to query the model with the mandatory fields (phase, prompt, reason and evidence_row). This is followed up with an automatic validator to check that the response is of the correct phase (or chooses to wait) and provides the evidence row given, is the correct length and doesn't include any prohibited words/phrases (code blocks, your tests failed etc). If the prompt fails then it is rejected and replaced with a deterministic reflection question, which is the same as the evidence prompt. Additionally, there is a deterministic claim checker to score the generated prompts for claims relating to outcome, feelings, code and unfounded numbers. This is for evaluation purposes and not a runtime gate.

E. Delayed Transfer Rubric (G3)

A transfer task is given to identify if the student is performing well temporarily with assistance or learning long term. The learners must write a program for a transfer task independently (no conversational/generative AI). This is marked using a 4 point rubric:

Each transfer task has a score between 0 and 4. One point is given for restating the goal, creating a test case, handling the edge case and returning a correct value. This is averaged over 4 transfer tasks. The rubric was validated using 120 simulated learners per run (5 seeds). Originally the transfer task had a 7 day delay with human study (out of scope).

V. Experiments

A. Data Split

2,598,189 events from a real dataset from a CS1 MOOC [14] were used. 24 learners with a common prefix of programming problems. We made sure there was no data leakage by keeping a strict split:

Development Split (16 learners): We used 1,871,292 events for development. The values for $\tau_{\text{pause}} = 20$s, cooldown = 180s were not optimised. Cooldown is an engineering default value from Phase 1 and τ_{pause} is the same as the pause time used in the metrics. Defaults were set during reviewing all learners in Phase 1, including those used in this test.

Held Out Test Split (8 learners): 726,897 events were used for learners {2, 5, 8, 11, 14, 17, 20, 23}. This equates to 966 learner questions and 10,687 observable decision points.

B. Baseline Policies

We compared TrajectoryCoach to five different policies:

1) *Rule (uncapped):* Always triggers the default scaffold at each decision point. No budgeting and no time gate.
2) *Phase 1 Policy:* Proactive triggering with prompt time caps. No idle time gate and no minimum evidence.
3) *On Demand (At Pauses):* Only triggered by learners. When a natural pause occurs (>20s). We simulate pauses using a rate from the base paper of 3 seeds.
4) *On Demand (stuck):* Only triggered by learners but weighted towards times of multiple runs.
5) *On Demand (Uniform Random):* Triggered randomly.

The on demand policy from the base paper was used. TrajectoryCoach provides deterministic budgeting, an idle time gate and evidence based validation.

VI. Evaluation

A. Main Metrics

We think of four main aspects of policies:

Interruption Rate (↓): This measures the percentage of prompts that are interrupted in the 5 seconds before and after the prompt (within a typing burst). We want this metric to be as low as possible as it will minimise disruption to the students thinking process.

Pause Alignment (↑): The percentage of scaffolds that are aligned to a pause of 20s or above.

Coverage (↑): The percentage of all valid decision points covered within 30 seconds.

Deliberation shift (Δ): The mean difference between the ratio of edits to (edits + runs) in the 300 seconds after the prompt and 300 seconds before the prompt. This is indicative of when the 'juncture' was replayed in old logs rather than behaviour change due to the prompt.

B. Significance

Confidence intervals were calculated using cluster bootstrapping. 1000 iterations were done. Student level resampling was performed and student trajectories were clustered.

VII. Experimental Results

A. RQ1: Key Results

The main results can be seen in Table I. A 6-fold decrease from best on demand, TrajectoryCoach achieved an interruption rate of 0.021 (95% CI [0.013, 0.028]). This is a 32-fold decrease from the ungated Phase 1 policy (0.657).

Figure 2 shows how the balance of non-interruptiveness and coverage of decision points. The uncapped rule baseline has high coverage (0.793) but also high interruptions (0.274). The on demand policies are very non-interruptive (0.127) but have low coverage (0.019) as novices rarely ask for help. The sweet spot is found with a total of 2038 timely scaffolds. This has a coverage of 0.137 and interruptions of 0.021.

Development learners: After adding the time gate and evidence floor, all experiments were reran with no changes to the 16 development learners (1,871,292 events). The trends are similar with the full system having low interruptions (0.0125) (95% CI [0.009, 0.017]), compared to the on demand help at pauses (0.068) +/- 0.006 and the Phase 1 policy (0.675). Without the time gate the interruptions increase to 0.285. Without the evidence floor junction precision decreases

TABLE I

MAIN RESULTS ON THE HELD-OUT TEST SPLIT (726897 EVENTS, 966 LEARNER–QUESTION UNITS, 10687 OBSERVABLE DECISION POINTS). ON-DEMAND ROWS ARE THE MEAN AND STANDARD DEVIATION OVER THREE SEEDS; THE OTHER ROWS ARE DETERMINISTIC. LOWER IS BETTER FOR INTERRUPTION. NO LEARNER IN THE LOGS SAW A SCAFFOLD, SO THE DELIBERATION COLUMN DESCRIBES THE MOMENTS A POLICY SELECTS, NOT AN EFFECT OF PROMPTING.

	Condition	Scaffolds	Interruption	Pause-aligned	Coverage	Deliberation Δ
E1	Rule baseline (uncapped)	10687	0.274	0.230	0.793	-0.0138
E2	Base paper, on-demand at pauses	461 ± 11	0.127 ± 0.014	0.355 ± 0.039	0.019 ± 0.001	-0.0068 ± 0.0025
E2	Base paper, on-demand when stuck	488 ± 13	0.837 ± 0.026	0.070 ± 0.003	0.019 ± 0.000	-0.0070 ± 0.0020
E2	Base paper, on-demand uniform	484 ± 11	0.829 ± 0.030	0.072 ± 0.009	0.019 ± 0.001	-0.0104 ± 0.0042
E3	**Proposed system**	2038	0.021	0.448	0.137	0.0079

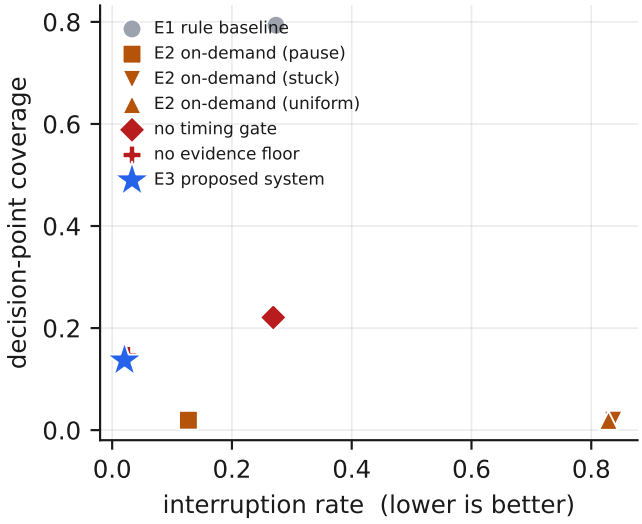


Fig. 2. Interruption against decision-point coverage on the test split. No configuration is best on both axes.

to 0.543 (from 0.696). Coverage is 0.136. We have used development learners to inform the Phase 1 defaults therefore not entirely independent.

B. RQ1: Time Gate Effectiveness

It can be seen that the time issue (Gap 1) was overcome by waiting 20s for the candidate to prompt. As can be seen in Table III, the time gate prevented 3269 candidates from scaffolding. Of these 2038 were scaffolded after 20s, 1060 were replaced by new candidates and 171 were dismissed after 600s with no pause. This means there was a change in the amount of deliberation (+0.0079). However the confidence interval includes 0 ([−0.0016, +0.0172]) since we are measuring the effectiveness of the junctions not the students.

C. RQ2: Groundedness and Hallucination

To test groundedness (Gap 2) the 100 identical prefixes were run with live generation. We used openai/gpt-oss-20b with temperature set to 0.0 and seed 0. Using prompt v2 55 (53 for v1) of the 100 unconstrained scaffolds made invalid claims. This was typically about outcomes (code works/won't work, occasionally counts and occasionally other feelings based claims). However with prompt v1 and the JSON schema

TABLE II

HUMAN RATING OF 30 SCAFFOLDS FROM THE FULL SYSTEM, BY THE TWO AUTHORS INDEPENDENTLY. LINEARLY WEIGHTED κ IS 0.328 FOR TIMING AND 0.343 FOR RELEVANCE, WHICH IS FAIR RATHER THAN STRONG AGREEMENT. THE SHEET SHOWED RATERS THE PHASE AND THE PROMPT TEXT BUT NOT WHEN THE SCAFFOLD WAS DELIVERED, SO THE TIMING COLUMN IS REPORTED FOR COMPLETENESS AND NO CLAIM RESTS ON IT.

Rater	Timing	SD	Relevance	SD
Rating Sheet Minahil	3.47	0.57	3.30	0.60
Rating Sheet Areeba Rated	3.47	0.63	3.47	0.51
Pooled	3.47	0.49	3.38	0.45

constraints only 14.0% were flagged. With prompt v2 (written after seeing what words were flagged for v1) none were flagged (Table V). Both runs had no rejected responses. This is because the prompt wording is different. There were only 26 unique questions in the 100 v2 prompts and the model never called any of the tools.

To see if the prompts are truly helpful 30 full system scaffolds were rated by both co-authors with a 4 point scale for time and 4 point scale for relevance (Table II). There was good agreement for the time (0.328) and relevance (0.343) (linear weighted Cohen's κ). The overall relevance score was 3.38 out of 4.0, showing that the prompts are still fairly relevant. The raters could not see the timing of the scaffolds as they only saw what phase it was in and what the prompt said. They could not see when the scaffolds were delivered.

D. RQ3: Validating the Delayed Transfer Rubric on Simulated Learners

We have found that performing the task at the point is a noisy signal of understanding (Gap 3). There is a strong correlation between assisted completion and latent skill ($\rho = 0.724$). However our unassisted transfer score is higher ($\rho = 0.793$) +/- 0.036 (Spearman). There is only a slight difference between the two. This is because in a human study learners would have to demonstrate decomposition of the problem on a later problem without any support.

VIII. ABLATION AND EFFICIENCY

A. Breakdown

An ablation was performed on the held out portion of the test set (Table IV and Figure 4). This demonstrates:

TABLE III

GAP-RESOLUTION TABLE. "BASE PAPER RESULT" IS OUR ADAPTATION OF THE BASE PAPER'S INTERACTION POLICY RUN ON OUR DATA, NOT A NUMBER REPRODUCED FROM THE PUBLISHED PAPER, WHICH REPORTS NEITHER OF THESE MEASURES.

Gap	Method / component	Metric	Base paper result	Our result	Addressed?
Gap 1: timing	Timing gate: hold a scaffold until a 20 s pause	Interruption rate on held-out prefixes	0.127 ± 0.014 (on-demand at pauses)	0.021	Partially
Gap 2: grounding	Evidence floor, JSON schema, validator, read-only tools	Share of scaffolds containing an unsupported claim	0.550 (unconstrained, same model)	0.000 (v2); 0.140 (v1)	Fully
Gap 3: transfer	Delayed no-AI transfer task and four-point rubric	Spearman r with latent skill, simulated learners	0.724 (assisted completion)	0.793 ± 0.036	Partially

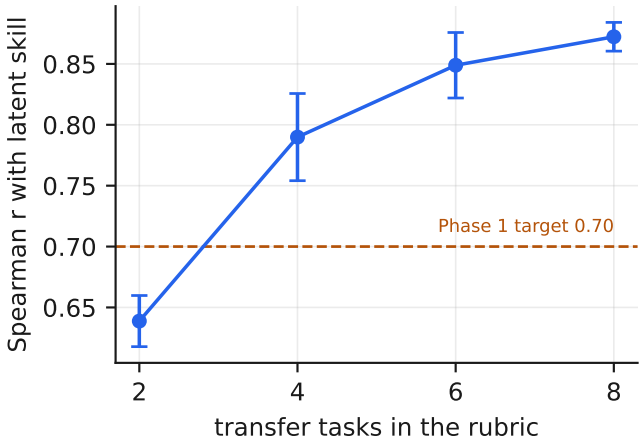


Fig. 3. Transfer-score correlation with latent skill against the number of tasks in the rubric, over five seeds. Simulated learners only.

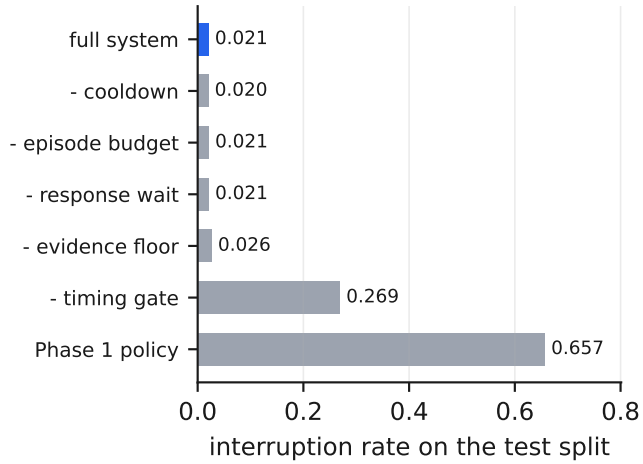


Fig. 4. Interruption rate with one component removed at a time.

TABLE IV

ABLATION ON THE TEST SPLIT. EACH ROW REMOVES ONE COMPONENT FROM THE FULL SYSTEM. THE TIMING GATE ACCOUNTS FOR ALMOST ALL OF THE REDUCTION IN INTERRUPTION; REMOVING THE RESPONSE WAIT AS WELL AS THE EPISODE BUDGET GIVES THE SAME RESULT AS REMOVING THE BUDGET ALONE, SO THE RESPONSE WAIT ADDS NOTHING ONCE THE COOLDOWN IS ACTIVE (THE COOLDOWN IS 180 S AND COVERS THE 120 S WAIT).

Configuration	Scaffolds	Interruption	Pause-aligned	Coverage
Full system	2038	0.021	0.448	0.137
− cooldown	2129	0.020	0.449	0.144
− episode budget	2955	0.021	0.444	0.206
− budget and response wait	2955	0.021	0.444	0.206
− evidence floor (G2)	2804	0.026	0.424	0.146
− timing gate (G1)	2364	0.269	0.310	0.221
Phase 1 policy, no components	3828	0.657	0.168	0.341

TABLE V

EFFICIENCY AND RELIABILITY (E6). THE OFFLINE ROWS REPLAY THE FULL TEST SPLIT ON ONE CPU CORE WITH NO NETWORK. COLUMNS FOR THOSE ROWS ARE: RUNTIME (S), EVENTS PER SECOND, PEAK MEMORY (MB), SCAFFOLDS PER 1000 EVENTS.

Condition	measures			
Rule baseline	24.7	29 392	7.7	14.70
Proposed system	24.8	29 366	1.5	2.80
Live model: openai/gpt-oss-20b, temperature 0.0, seed 0				
Constrained, prompt v1	101	56 546	0.74	0.000
Constrained, prompt v2	100	77 627	0.68	0.000
Unconstrained generation	100	38 850	0.75	–
Columns for the live rows are: API calls, total tokens, mean latency (s), validator rejection rate.				

1) Without the time gate (− timing gate) the interruptions increased by around 13 times (0.269) and the pause alignment reduced by 0.138 (0.310). This shows that the vast majority of the reduction in disruption comes from the time gate.
2) Without the episode budget and wait time (2955 scaffolds) the interruptions were the same as when only the episode budget was removed. The wait time was not necessary as the 120 second response wait was included within the 180 second cooldown.
3) Evidence Floor Trade-off: Without the evidence floor the amount of scaffolds delivered increased from 2038 to 2804. However the accuracy of the junction dropped from 0.717 to 0.559. The amount of flagged prompts increased from 0.14 to 0.39 in live runs (prompt v1 - 2 samples of 100 prefixes - indicative only).

B. Cost and Efficiency

The efficiency can be seen in Table V. TrajectoryCoach can handle 29366 events per second on one core. The maximum memory usage during offline replay was 1.5MB. The average response time was 0.74 (0.68 for v1) seconds with an average of ∼560 tokens per call (∼776 for v2). The model is also

TABLE VI
FAILURE CATEGORIES OVER THE 2038 SCAFFOLDS THE FULL SYSTEM DELIVERED ON THE TEST SPLIT. CATEGORIES ARE NOT EXCLUSIVE; 190 SCAFFOLDS (0.093) CARRY AT LEAST ONE FLAG.

Category	Cases	Share of delivered
superseded	1060	0.520
suppressed submission	466	0.229
held then dropped	171	0.084
late delivery	92	0.045
abandoned after	59	0.029
interrupted burst	42	0.021

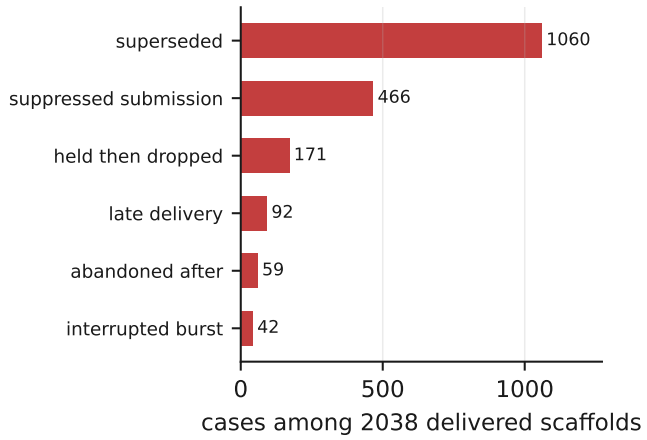


Fig. 5. Failure categories among delivered scaffolds on the test split.

required to word each candidate so there is a low delivery rate of 2.80 scaffolds per 1000 events. However there were around 4.5 calls per 1000 events (3269 candidates were held per 726,897 events). This is around 2500 tokens per 1000 events for v1.

IX. FAILURE ANALYSIS

A. Failure Taxonomy

Operational failures can be broken down into 6 categories. There were 190 (9.32%) scaffolds with a delivery flag (late, abandoned or interrupted) or (the other 3 categories show scaffolds or submissions that did not result in a delivery) out of 2038 deliveries (scaffolds) (results/phase2/failure_analysis.json). The main failures can be seen in Table VI. These are shown in Figure 5. The percentages below show what percentage of the 2038 delivered scaffolds this represents.

Superseded Junctions (52.0% / 1060): Fast editing meant that the scaffold in the buffer could be at the 20 second junction. However a new candidate junction may have superseded this. Proposed solution (not implemented): Priority Queuing (evaluate, monitor and plan).

Suppressed Submissions (22.9% / 466): 466 of the 1205 submissions (38.7%) from the student were not scaffolded by an evaluate action. This was either because the student had used all 3 prompts or were on cooldown. This means that there was an opportunity missed to evaluate. Proposed solution (not implemented): Separate budget for submissions.

Held and Dismissed (8.4% / 171): The learner failed to pause for at least 20 seconds across the 600 second horizon.

Late Delivered (4.5% / 92): The gate delayed the scaffold for over 120 seconds.

Abandoned (2.9% / 59): No further events were generated by the learner within 30 minutes of receiving the scaffold.

Interrupted Typing Burst (2.1%): The student paused for 20 seconds and were scaffolded. However they began typing again within 5 seconds (42 occurrences).

B. Example Traces from the Replay Logs

Below are examples of successful interventions:

Trace 1: Evaluation Scaffold: Learner 2: Row 289,983. Phase: Evaluate. Reason: Submission observed (correctness unknown). Prompt: A submission was recorded. What evidence would you use to check the correctness? Include edge cases. Window: 40 events. (2 runs, 27 edits). Scaffolds were held for 20 seconds. Next event was generated by the learner 43 seconds after delivery.

Trace 2: Monitoring Scaffold: Learner 2: Row 351,346. Phase: Monitor. Reason: Multiple runs in recent window. Prompt: Recent window has 3 runs. What hypothesis are you testing? What would confirm your hypothesis? Window: 65 events. (3 runs, 48 edits). Delivered after 20 seconds. Next event was generated by the learner 1 second after delivery.

Trace 3: Abandoned: Learner 2: Row 292,689. Phase: Evaluate. 138 events in window. (1 run, 131 edits). Scaffolds were delivered after submission to window. Next event was recorded 16.3 hours later (58,614s).

Trace 4: Learner 11: Row 1059512. Phase: Plan. Failure: Late Delivered (Stale). First edit triggered the planning prompt, however the learner continued to type. The prompt was not delivered until 177 seconds later. This meant that the prompt was no longer a good planning question as it was long after the initial edit.

X. LIMITATIONS AND THREATS TO VALIDITY

In this section we want to highlight some limitations.

1. No Live Classroom Deployment The data was fully post-hoc. As such it is not representative of students' logs, and the shift in deliberation ($\Delta = +0.0079$) and interruption rate are reflective of the selectivity of the policy, rather than students' learning.
2. A small held out test set Only 8 students were part of the held out test set. Had there been more students they may have acted differently, and therefore the optimal pause threshold may have changed. There were 726,897 events and 966 learner question units.
3. Delayed Transfer Cohorts Furthermore, the delayed transfer tool was used with synthetic students, and not actual exam scores.
4. Not using fine grained telemetry TrajectoryCoach will not consider source code, compiler traces or unit test

assertions. This will ensure privacy but only allows for process level metacognition.

5. Human annotation was done by the co-authors independently. It was not done blindly due to the delay in delivery giving away the condition. There was a fair level of agreement ($\kappa = 0.343$) and reflects our ideas about the purpose of the system.
6. Word and numbers list. 0 flagged doesn't mean none had any, but none were flagged.
7. Prompt v2 was designed based on the words that were flagged on prompt v1.
8. Only 26 unique questions in the 100 v2 prompts.
9. The model didn't call any of the 4 tools.
10. Only one model, one seed, 100 prefixes.
11. On demand comparator is our implementation with simulated requests.
12. Defaults for phase 1 (180s, 3 prompts, 300s) were chosen based on observing all 24 learners including those in the test set.
13. Design bias. No tuning of thresholds were performed on test learners. The timing gate and evidence floor were both implemented after looking at the results from the test set. The values (20s, 5 events) were equivalent to the hard coded thresholds in the metrics and weren't swept. Re-running on the 16 development learners shows the same trend (Section VII-A).

XI. Conclusion and Open Science

The pedagogic agent suggested here can be automated to provide timely metacognitive support to the user. The TrajectoryCoach tool demonstrates this in replay, by reducing the interruption rate to 0.021 (compared to 0.127 for on demand and 0.657 for phase 1 policy (0.269 without the timing gate)). However, this comes at the cost of coverage (0.137 compared to 0.793 for the uncapped rule). This pattern is seen on the development learners (0.0125). Of the 100 prompts generated using prompt v1, 14 were flagged for unsupported claims, compared to 53 of the 100 unbound prompts. Using prompt v2, 0 were flagged (compared to 55 of 100). However, prompt v2 was designed based on what was flagged on prompt v1, and only had 26 unique questions. The model also didn't call any tools. When using the simulated learners, the delayed transfer score had a higher correlation with the latent skill ($\rho = 0.793$) than the assisted completion ($\rho = 0.724$), however there was not a large gap between them. None of these results show any learning effect. An important design principle for educational AI systems is Restraint. Knowing when to stay quiet is as important as knowing what to say. Future work includes a human study, an integration with the live editor and real timer, and a separate budget for submissions.

The release tag phase2-final containing all of the codes, ablation scripts, log processors and evaluation logs can be found on Github (https://github.com/Areeba444/TrajectoryCoach).

References

[1] T. Phung, H. Choi, M. Wu, A. Singla, and C. Brooks, "Plan more, debug less: Applying metacognitive theory to AI-assisted programming education," in *Artificial Intelligence in Education (AIED 2025)*, ser. LNAI, vol. 15877, 2025, pp. 3–17.

[2] X. Hou, Z. Liu, G. Wang, and N. Shi, "Bridging awareness and behavior: Decoding implicit metacognitive behaviors in AI-assisted programming via fine-grained log analysis," *Frontiers in Psychology*, vol. 17, p. 1790196, 2026.

[3] C. C. Y. Yang, M. Li, and A. Y. Q. Huang, "Leveraging large language models to enhance self-regulated learning in programming education with explainable AI," *Journal of Computer Assisted Learning*, vol. 42, no. 2, p. e70206, 2026.

[4] P. Bassner, B. Lenk-Ostendorf, R. Beinstingel, T. Wasner, and S. Krusche, "Less stress, better scores, same learning: The dissociation of performance and learning in AI-supported programming education," *Computers and Education: Artificial Intelligence*, vol. 10, p. 100537, 2026.

[5] Y.-M. Yan, C.-Q. Chen, Y.-B. Hu, and X.-D. Ye, "LLM-based collaborative programming: Impact on students' computational thinking and self-efficacy," *Humanities and Social Sciences Communications*, vol. 12, p. 149, 2025.

[6] H. Heickal and A. Lan, "A classroom study of LLM-generated feedback intervention in introductory programming," in *Impactful and Responsible AI Systems for Education Workshop*, ser. Proceedings of Machine Learning Research, vol. 339, 2026, pp. 12–21. [Online]. Available: https://proceedings.mlr.press/v339/heickal26a.html

[7] P. Silva and E. Costa, "Assessing large language models for automated feedback generation in learning programming problem solving," in *Innovation and Responsibility in AI-Supported Education Workshop*, ser. Proceedings of Machine Learning Research, vol. 273, 2025, pp. 116–124. [Online]. Available: https://proceedings.mlr.press/v273/silva25a.html

[8] A. Durg, C. Kultur, A. Zhang, and J. Savelka, "LLM-powered framework for automatic generation of metacognitive scaffolding cues for introductory programming in higher education," in *Sixth International Workshop on Intelligent Textbooks*, ser. CEUR Workshop Proceedings, vol. 4010, 2025, pp. 25–36. [Online]. Available: https://ceur-ws.org/Vol-4010/itb25_s1p3.pdf

[9] J. Perretta, A. DeOrio, A. Guha, and J. Bell, "Instructor-written hints as automated test suite quality feedback," in *Proceedings of the 56th ACM Technical Symposium on Computer Science Education (SIGCSE TS 2025)*, vol. 1, 2025.

[10] G. Pitts, A. P. Hridi, and A. B. Lekshmi Narayanan, "A survey of LLM-based applications in programming education: Balancing automation and human oversight," in *Fourth Workshop on Bridging Human-Computer Interaction and Natural Language Processing*, 2025, pp. 255–262.

[11] J. Leinonen and A. Hellas, "IDE action log dataset from a CS1 MOOC," Zenodo, 2022, licensed CC BY 4.0.

[12] S. T. Iqbal and B. P. Bailey, "Leveraging characteristics of task structure to predict the cost of interruption," in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI 2006)*, 2006, pp. 741–750.

[13] C. Parnin and S. Rugaber, "Resumption strategies for interrupted programming tasks," *Software Quality Journal*, vol. 19, no. 1, pp. 5–34, 2011.

[14] G. Mark, D. Gudith, and U. Klocke, "The cost of interrupted work: More speed and stress," in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI 2008)*, 2008, pp. 107–110.

[15] B. J. Zimmerman, "Becoming a self-regulated learner: An overview," *Theory Into Practice*, vol. 41, no. 2, pp. 64–70, 2002.

[16] D. Wood, J. S. Bruner, and G. Ross, "The role of tutoring in problem solving," *Journal of Child Psychology and Psychiatry*, vol. 17, no. 2, pp. 89–100, 1976.

[17] V. Aleven, E. Stahl, S. Schworm, F. Fischer, and R. Wallace, "Help seeking and help design in interactive learning environments," *Review of Educational Research*, vol. 73, no. 3, pp. 277–320, 2003.

[18] T. Sinha and M. Kapur, "When problem solving followed by instruction works: Evidence for productive failure," *Review of Educational Research*, vol. 91, no. 5, pp. 761–798, 2021.

[19] R. McCauley, S. Fitzgerald, G. Lewandowski, L. Murphy, B. Simon, L. Thomas, and C. Zander, "Debugging: A review of the literature from an educational perspective," *Computer Science Education*, vol. 18, no. 2, pp. 67–92, 2008.

[20] M. C. Jadud, "An exploration of novice compilation behaviour in BlueJ," Ph.D. dissertation, University of Kent, 2006.

[21] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, 2023.

[22] B. T. Willard and R. Louf, "Efficient guided generation for large language models," 2023, arXiv:2307.09702.

[23] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," 2023, arXiv:2210.03629; ICLR 2023 camera-ready version.

[24] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J.-R. Wen, "A survey on large language model based autonomous agents," *Frontiers of Computer Science*, 2024.

[25] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging LLM-as-a-judge with MT-Bench and chatbot arena," 2023, arXiv:2306.05685; NeurIPS 2023 Datasets and Benchmarks Track.

[26] G. Kiczales, J. Lamping, A. Mendhekar, C. Maeda, C. Lopes, J.-M. Loingtier, and J. Irwin, "Aspect-oriented programming," in *ECOOP'97 – Object-Oriented Programming*, ser. Lecture Notes in Computer Science, vol. 1241. Springer, 1997.

[27] S. Slade and P. Prinsloo, "Learning analytics: Ethical issues and dilemmas," *American Behavioral Scientist*, vol. 57, no. 10, pp. 1509–1528, 2013.